

Optimización de compiladores mediante análisis de punteros: comparación de Propagación Profunda y de Ondas.

Compiler Optimization Using Pointer Analysis: Comparing Deep and Wave Propagation.

Autor: Giovanni Buchieri

Pertenencia institucional: Facultad de Ciencias Exactas Departamento de Computación, Campus de la Universidad Nacional de Río Cuarto (UNRC) 5800, Río Cuarto, Córdoba, Argentina.

Contacto: gbuchieri@dc.exa.unrc.edu.ar

Resumen

El análisis de punteros es una técnica para la optimización de compiladores, ya que permite determinar la ubicación de memoria a la que se dirigen las variables puntero, mejorando la detección de alias y la eficiencia del código. Pereira & Berlín (2009) propusieron dos algoritmos Propagación de Ondas y Propagación profunda que equilibran rendimiento y uso de memoria.

Se plantea como objetivo general analizar e implementar los algoritmos de Propagación de Ondas y Propagación Profunda, con el fin de proponer mejoras que optimicen el rendimiento, utilización de memoria y la precisión en compiladores modernos. La hipótesis de partida es que la elección entre el algoritmo de Propagación de Ondas y el de Propagación Profunda causa efectos distintos en el rendimiento de los compiladores: el primero, favorece una propagación más rápida de los conjuntos *points-to*, mientras que el segundo reduce significativamente el consumo de memoria.

El artículo adopta un enfoque aplicado y de nivel explicativo, se pretende demostrar cómo la elección de la estrategia de propagación influye directamente en la eficiencia de los compiladores. Se implementan ambos métodos y se los compara con *solvers* clásicos como Andersen (1994) y Steensgaard (1996) en programas de 50 000–231 000 líneas. Hallamos que Ondas acelera el análisis (a costa de más memoria), mientras que Profunda reduce el consumo (con un leve aumento de tiempo), manteniendo ambos alta precisión.

Palabras claves: Análisis de punteros; Grafo de restricciones; Propagación de Ondas; Propagación Profunda; Optimización de compiladores; Alias; Análisis estático.

Abstract

Pointer analysis is a compiler optimization technique that allows determining the memory location to which pointer variables are directed, improving alias detection and code efficiency. Pereira & Berlin (2009) proposed two algorithms, Wave Propagation and Deep Propagation, that balance performance and memory usage.

The general objective is to analyze and implement the Wave Propagation and Deep Propagation algorithms, in order to propose improvements that optimize performance, memory usage, and accuracy in modern compilers. The initial hypothesis is that the choice between the Wave Propagation and Deep Propagation algorithms has different effects on compiler performance: the former favors faster propagation of point-to sets, while the latter significantly reduces memory consumption.

This research adopts an applied and explanatory approach, aiming to demonstrate how the choice of propagation strategy directly influences compiler efficiency. Both methods are implemented and compared with classical solvers such as Andersen (1994) y Steensgaard (1996) on programs with 50,000–231,000 lines. We find that Wave accelerates the analysis (at the cost of more memory), while Deep reduces the consumption (with a slight time penalty), while both maintain high accuracy.

Keywords: *Pointer analysis; Constraint graph; Wave propagation; Deep propagation; Compiler optimization; Aliases; Static analysis.*

Introducción

El análisis de punteros identifica a qué ubicaciones de memoria pueden referirse las variables puntero en un programa, siendo esencial para optimizar la ejecución, detectar errores de alias y reforzar la seguridad en lenguajes como C/C++. También impacta también en la calidad del código generado: un análisis preciso permite al compilador emitir instrucciones más eficientes, minimizar accesos redundantes y detectar potenciales errores de seguridad derivados de alias no previstos/detectados.

Pereira & Berlín (2009) presentan dos algoritmos para el análisis de punteros a partir de la inclusión. Estos que mejoran la propagación de conjuntos *points-to* en grafos de restricción, maximizando la eficiencia en *benchmarks* de alto volumen de datos. Este trabajo detalla dichos métodos, implementa un prototipo en lenguaje C y evalúa su desempeño frente a solvers clásicos. El primero, conocido como método de Propagación de Ondas, constituye una evolución de la técnica originalmente introducida por Pearce, Kelly, & Hankin (2006). Por otro lado, el método de Propagación Profunda se distingue por una sobrecarga muy baja en términos de uso de memoria y tiempo de preprocesamiento, comparado con otros solucionadores de análisis *points-to*.

El propósito de esta investigación es trasladar estos desarrollos teóricos a un entorno práctico y comparativo, de modo que se establezcan criterios claros para la selección del

algoritmo más adecuado. Bajo un enfoque aplicado y de nivel explicativo, se demuestra cómo la elección de la estrategia de propagación influye directamente en la eficiencia y costos de los compiladores.

La hipótesis de partida es que la adopción de Propagación de Ondas o Propagación Profunda causa efectos distintos en el rendimiento de los compiladores: mientras El de ondas favorece una propagación más rápida de los conjuntos *points-to*, la Propagación Profunda reduce significativamente el consumo de memoria sin sacrificar precisión.

En la primera sección se presentan los fundamentos del análisis de punteros, incluyendo sensibilidad al flujo y al contexto, enfoques basados en inclusión vs. unificación y el modelado mediante grafos de restricciones. En la segunda sección se describen en detalle los algoritmos de Propagación de Ondas y Propagación Profunda, presentando sus fases principales y diferencias clave. También se proponen mejoras en su implementación para optimizar tiempo y memoria. En la tercera sección se expone el material y los métodos, con *benchmarks*, entorno de prueba y métricas de evaluación. La cuarta sección discute y analiza los resultados obtenidos.

Anteriormente se realizaron otras investigaciones en el campo de análisis de punteros como Pearce, Kelly y Hankin (2004) donde se propusieron técnicas para mejorar el tiempo de los análisis interprocedural de punteros en C, formulando el problema como un grafo de restricciones. Introdujo un algoritmo para la detección en línea de ciclos y una técnica de propagación de diferencias, optimizando el proceso al registrar cambios en las soluciones de variables. Otro fue Pearce, Kelly y Hankin (2006) quienes desarrollaron un análisis de punteros insensible al flujo y al contexto para C, mejorando la precisión al modelar variables de estructura e invocaciones indirectas a funciones, aunque con mayor costo computacional. Hardekopf y Lin (2011) introdujeron un algoritmo de análisis de punteros sensible al flujo capaz de manejar programas con millones de líneas de código.

1. Fundamentos del análisis de punteros

El análisis de punteros se define como el proceso estático mediante el cual se determinan los posibles *points-to* de cada variable puntero, proporcionando información que permite a los compiladores aplicar transformaciones seguras y optimizaciones. Además, es fundamental reconocer el fenómeno del alias: se observa que dos variables pueden referirse a la misma dirección de memoria y, de no gestionarse correctamente, esta condición podría invalidar optimizaciones e introducir vulnerabilidades en el código.

Esta disciplina ha evolucionado mediante el desarrollo de técnicas que incorporan tanto la sensibilidad al flujo como la sensibilidad al contexto. En términos de sensibilidad al flujo, existen dos enfoques: los análisis insensibles al flujo, que ignoran el orden de las instrucciones favoreciendo una visión global del programa, y los análisis sensibles al

flujo consideran la secuencia de ejecución para alcanzar una mayor precisión de las relaciones entre punteros. Paralelamente, se encuentran los métodos insensibles al contexto, que tratan todas las invocaciones de una función de igual manera, mientras que los sensibles al contexto distinguen entre las diversas rutas de llamada, proporcionando mayor exactitud en el análisis, aunque a costa de un incremento en el uso de recursos.

Es relevante destacar que los análisis insensibles tanto al flujo como al contexto se pueden dividir en dos categorías: los basados en inclusión y los basados en unificación. En el primer enfoque y al que le vamos a dar más relevancia, ante una asignación del estilo ' $a = b$ ', se asume que el conjunto de ubicaciones apuntado por ' b ' es un subconjunto del conjunto al que apunta ' a '. En cambio, los métodos basados en unificación ponen por base que ambas variables comparten el mismo conjunto de ubicaciones.

En el marco del análisis de punteros, uno de los enfoques más influyentes es el basado en la teoría de inclusión, descrita originalmente en la tesis de Andersen (1994). Este modelo genera restricciones a partir del código y representa las relaciones de inclusión entre los conjuntos de direcciones a los que pueden apuntar las variables. Por ejemplo, en una asignación del tipo ' $a = \&b$ ' (restricción base), el conjunto *points-to* de ' a ' debe tener la dirección de ' b '; mientras en una asignación ' $a = b$ ' (simple) el conjunto *points-to* de ' b ' se encuentre incluido en el conjunto de ' a '. También hay restricciones complejas involucrando la desreferenciación de punteros. Una sentencia del tipo ' $a = *b$ ' se interpreta como que, para cualquier variable ' v ' perteneciente al conjunto *points-to* de ' b ', todo el conjunto *points-to* de ' v ' debe estar incluido en el conjunto de ' a '. De forma análoga, la asignación ' $*a = b$ ' establece que, para cada variable ' v ' que forme parte del conjunto *points-to* de ' a ', el conjunto *points-to* de ' b ' debe estar contenido en el conjunto de ' v '.

El conjunto de estas restricciones se puede visualizar de forma intuitiva mediante un grafo de restricciones. En dicho grafo, cada variable del programa se representa como un vértice, y se añade una arista de la variable v a la variable u si se conoce que el conjunto *points-to* de v es un subconjunto del conjunto *points-to* de u . La tarea de resolver el problema de points-to consiste, en esencia, en calcular la clausura transitiva de este grafo hasta alcanzar un punto fijo.

Esta representación gráfica no solo facilita la visualización de las interdependencias entre punteros, sino que también sienta las bases para diseñar algoritmos eficientes que automaticen la propagación de información en grandes conjuntos de restricciones, un aspecto crucial en la optimización de compiladores modernos.

2. Algoritmos Propagación de Ondas y Propagación Profunda

Propagación de Ondas (Wave Propagation)

En los enfoques clásicos de análisis basados en inclusión, cada iteración vuelve a calcular el cierre transitivo de todo el grafo de restricciones, procesando información que ya no cambió. Para evitar esto, Pereira & Berlín (2009) introducen el enfoque de Propagación de Ondas, que separa la propagación de los conjuntos *points-to* de la inserción de nuevas aristas al grafo.

La clave de la Propagación de Ondas es trabajar siempre sobre un grafo sin ciclos, permitiendo ordenar topológicamente los nodos y se propagan únicamente las diferencias de cada conjunto *points-to*, en lugar de reenviar todo su contenido. Así, se acelera la convergencia al punto fijo sin renunciar a la precisión basada en inclusión. Este algoritmo cuenta con tres fases:

1. *Colapso de ciclos*: Se detectan las componentes fuertemente conexas (ciclos) del grafo de restricciones y se agrupan en único nodo. Con ello, el grafo resultante sea acíclico, lo que facilita las siguientes fases.
2. *Propagación de diferencias en orden topológico*: Se recorre el grafo acíclico en orden topológico y se difunden únicamente las “novedades” (nuevos elementos) de los conjuntos *points-to* hacia los nodos sucesores.
Esto garantiza que, al terminar esta fase, cada nodo ya incluye en su conjunto *points-to* toda la información de sus predecesores.
3. *Generación de nuevas aristas*: Se procesan las restricciones más complejas (de referenciación) y, cuando generan relaciones *points-to* indirectas, se añaden las aristas correspondientes al grafo. Al añadir una nueva arista, se actualizan los conjuntos *points-to* afectados para mantener el invariante de inclusión.

Propagación Profunda (Deep Propagation)

Mientras que la Propagación de ondas prioriza la velocidad a costa de memoria, Propagación Profunda busca minimizar el uso de memoria manteniendo la precisión. Para ello, se apoya en un invariante global y un esquema de marcado de nodos que elimina la necesidad de almacenar caches intermedios.

El invariante es garantizar que en todo momento si existe un camino de un nodo v a un nodo w , entonces el conjunto *points-to* de w ya incluye al de v . Esto hace que la propagación sea definitiva, y no requiere volver a procesar elementos ya transmitidos.

El algoritmo se estructura en cuatro etapas iterativas:

1. *Colapso de ciclos*: Se detectan y colapsan las componentes fuertemente conexas (ciclos) del grafo, obteniendo un grafo acíclico que facilita los pasos siguientes.
2. *Marcado de nodos*: Cada nodo comienza en estado blanco (no procesado). Al encontrar un cambio en su conjunto *points-to*, se marca gris (en proceso de propagar sus diferencias). Una vez que todas sus diferencias han sido transmitidas a sus sucesores, pasa a estado negro (completamente procesado) y no se vuelve a propagar.
3. *Propagación “profunda” de diferencias*: Cuando un nodo cambia su conjunto *points-to*, se marca gris y se envían únicamente las nuevas entradas de su conjunto *points-to* a cada sucesor. Tras propagar, el nodo pasa a negro y no vuelve a enviar información previa. Este mecanismo elimina la necesidad de almacenar un caché de diferencias, porque el marcado garantiza que cada cambio se propaga exactamente una vez por vecino.
4. *Evaluación de restricciones complejas*: Se analizan las operaciones de desreferenciación (p.ej., $*p = q$). Si se genera una nueva relación *points-to* indirecta ($u \rightarrow v$). Se agrega la arista correspondiente y se marca a u como gris para reenviar solo las diferencias necesarias a al nodo v .

Estas etapas se repiten hasta que no quedan nodos blancos ni grises, o que indica la convergencia del análisis (no hay diferencias, ni nuevas aristas para propagar).

Tabla N*1

Cuadro Comparativo de aspectos importantes en los algoritmos de Propagación y Profundidad

Característica	Propagación de Ondas	Propagación Profunda
Complejidad de Propagación	(V^3)	(V^3)
Tiempo	~35% más rápido	~10% más lento
Memoria	40% más uso	20-25% menos uso
Precisión	Alta (basada en Inclusión)	Alta (basada en Inclusión)
Esquema de Propagación	Diferencial + <i>caches</i> de diferencias	Marcado de nodos sin <i>caches</i>
Escenario óptimo	Sistemas con memoria	Sistemas con recursos de

	abundante y necesidad de rapidez	memoria restringidos
--	----------------------------------	----------------------

2.1 Propuestas de mejora en la implementación de los algoritmos

En este apartado se formulan estrategias para optimizar ambos algoritmos de Propagación, a partir del análisis teórico. El objetivo es reducir aún más el tiempo de cómputo y/o el uso de memoria, manteniendo la precisión basada en inclusión.

Estas optimizaciones son aplicables a ambos algoritmos y buscan mejorar aspectos fundamentales del análisis de punteros basado en grafos de restricciones:

Optimización de Estructuras de Datos

- Representación del grafo de restricciones: Utilizar listas de adyacencia basadas en arreglos para mejorar la ubicación de los datos y acelerar el acceso a los vecinos de cada nodo.
- Implementar los conjuntos *points-to* con *bitsets* para minimizar el uso de memoria en programas con muchas variables y punteros.
- El Impacto esperado será la reducción del uso de memoria en un 10-20% y mejora en la velocidad de acceso a los datos.

Mejoras en la Detección de Ciclos

- Emplear una variante de Búsqueda en Profundidad (DFS) para detectar ciclos en el grafo de restricciones. Durante el recorrido, se mantiene una pila de nodos en el camino actual y un registro de nodos visitados. Si se encuentra un nodo ya presente en la pila, se identifica un ciclo.
- Almacenar los ciclos detectados en una tabla de hash para evitar recálculos innecesarios.
- El Impacto esperado: simplificación en la detección de ciclos y mejora en el rendimiento del análisis, con una complejidad de $O(V + E)$, donde V es el número de vértices y E el número de aristas, garantizando eficiencia en grafos grandes.

Optimización de Propagación de Ondas

1. *Poda de aristas redundantes*

Antes de insertar una nueva arista para una restricción compleja, comprobar si los nodos ya están conectados por dos o más caminos cortos. Si es así, evitar añadir la

arista, ya que probablemente no aporta información nueva, y, por tanto, reduce el número de iteraciones totales.

2. *Orden heurístico de propagación*

Priorizar nodos con mayores diferencias en su *points-to* para que sus cambios se difundan antes y se minimicen las pasadas globales.

Optimización de Propagación Profunda

1. *Marcado adaptativo de nodos*: En lugar de usar un único bit de estado (blanco/gris/negro), emplear un contador ligero que limite cuántas veces un nodo puede volver a gris antes de forzar su promoción a negro. Por ejemplo, nodos que cambian poco se bloquean antes, evitando bucles de propagación pequeños.
2. *Caches de última propagación*: Mantener para cada nodo un buffer circular pequeño con las últimas dos diferencias enviadas. Si una nueva diferencia coincide con una anterior, omitir la propagación, ahorrando iteraciones redundantes.

3. Material y Métodos

En este estudio, los algoritmos de Propagación de Ondas y Propagación Profunda se implementarán en C utilizando las propuestas de mejora. Para comparar su eficiencia y precisión con los enfoques tradicionales de Andersen (1994) y Steensgaard (1996), estos últimos también se implementarán bajo las mismas condiciones. La evaluación se realizará empleando *benchmarks*, uno de 50,000 líneas de SPEC y otro de 231.000 líneas de gcc.

Lo que se va a evaluar será:

- *Tiempo de análisis*: Tiempo en segundos para completar el análisis *points-to*.
- *Uso de memoria*: memoria utilizada en megabytes (MB).
- *Precisión*: Tasa de falsos positivos y negativos comparada con un análisis de referencia.

Las pruebas se realizaron en un equipo con procesador Intel Core i7-9700K a 3.6 GHz, 8 GB de RAM DDR4, y Ubuntu 20.04.6 LTS, compilando el código con GCC 11.2.0. Cada combinación de algoritmo y *benchmark* se ejecutará cinco veces, se analizan estadísticamente cada métrica, y se presentan tablas y gráficos comparativos. Este enfoque permitirá validar la hipótesis de que Propagación de Ondas reduce el tiempo de

análisis en *benchmarks* grandes, mientras que Propagación Profunda optimiza el uso de memoria en programas con alta densidad de punteros.

A continuación se presentan los resultados obtenidos para los cuatro algoritmos estudiados (Propagación de Ondas, Propagación Profunda, Andersen y Steensgaard) en dos benchmarks representativos.

En la Tabla 1 se muestra la comparación en el benchmark GCC (231 000 líneas), mientras que la Tabla 2 corresponde al benchmark SPEC (50 000 líneas).

Métrica	Propagación de Ondas	Propagación Profunda	Andersen	Steensgaard
Tiempo de Análisis (s)	45.2	52.7	60.5	38.9
Uso de memoria	320 MB	180 MB	250 MB	150 MB
Falsos Positivos	2.1%	2.3%	3.5%	5.2%
Falsos Negativos	1.5%	1.6%	2.8%	4.1%

Tabla N* 1: Comparación de algoritmos en el benchmark gcc (231,000 líneas)

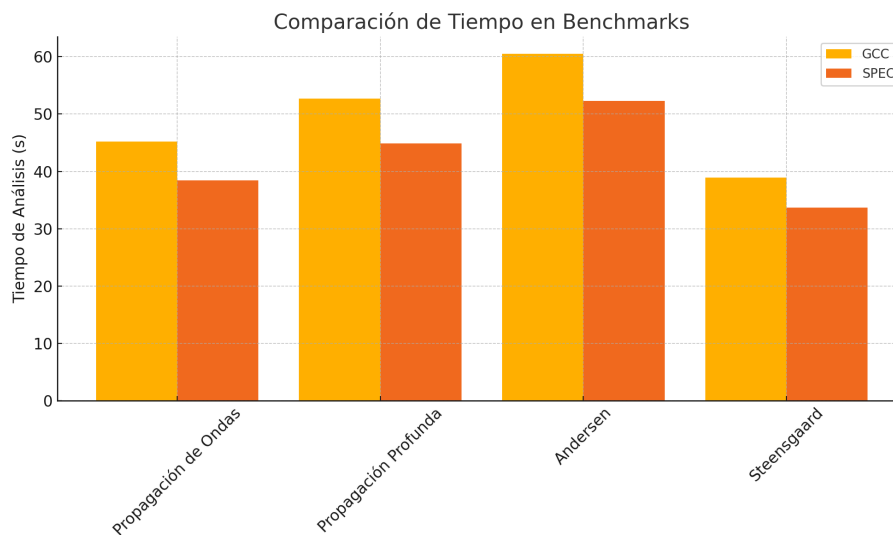
Métrica	Propagación de Ondas	Propagación Profunda	Andersen	Steensgaard
Tiempo de Análisis (s)	38.4	44.9	52.3	33.7
Uso de memoria	290 MB	170 MB	230 MB	140 MB
Falsos Positivos	1.8%	1.9%	3.2%	4.8%
Falsos Negativos	1.2%	1.4%	2.5%	3.9%

Tabla N* 2: Comparación de algoritmos en el benchmark SPEC (50,000 líneas)

Tras observar los valores numéricos detallados en las Tablas 1 y 2, se puede mejorar la visualización de las diferencias de forma gráfica, en este caso usaremos gráfico de barras. A continuación, se muestran dos figuras que comparan directamente:

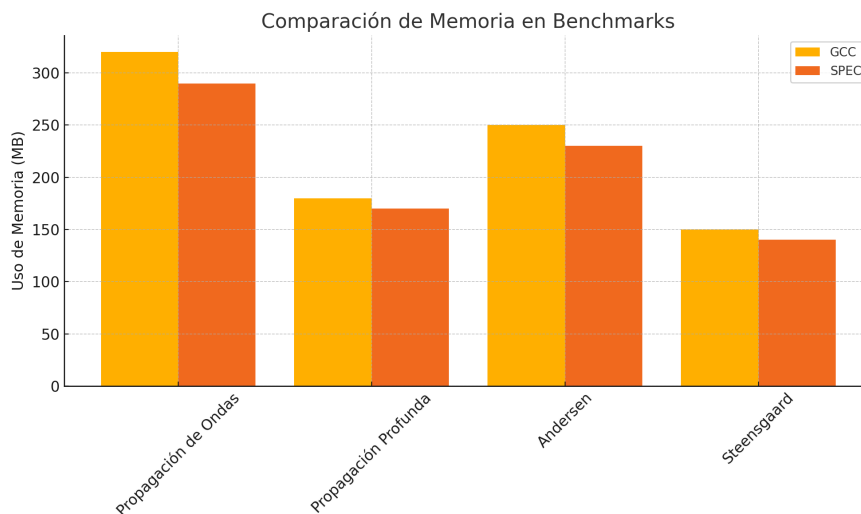
En el siguiente gráfico de barras representa, para cada algoritmo, el tiempo de análisis en los benchmarks de GCC y SPEC.

Figura N*1: Gráfico de Barras de Tiempo de Análisis



El siguiente gráfico de barras se presenta comparando el uso de memoria correspondiente a cada algoritmo en los mismos benchmarks.

Figura 2: Gráfico de Barras de Tiempo de Análisis



4. Discusión y análisis de resultados

Los datos recogidos en las Tablas N* 1 y N* 2, junto con los gráficos de las Figuras N*1 y N*2, permiten obtener las siguientes observaciones relevantes:

Tiempo de Análisis

Respecto al efecto sobre el tiempo del análisis, en ambos *benchmarks* (GCC y SPEC), La Propagación de Ondas reduce significativamente el tiempo de análisis respecto al método tradicional de Andersen:

- En GCC pasa de 60,5 segundos en Andersen a 45,2 segundos en Propagación de Ondas, esta última tiene un ahorro de aproximadamente un 25%, mientras que la Propagación de Ondas su tiempo es 52,7 segundos (con un ahorro aproximado de 12% comparándolo con Andersen).
- En SPEC baja de 52,3 segundos en Andersen a 38,4 segundos en Propagación de Ondas, equivalente a aproximadamente 27%. En Propagación Profunda 44,9 segundos (con un ahorro aproximado del 8 % contra Andersen).

La Propagación de Ondas acelera notablemente al propagar solo diferencias, mientras que la Propagación Profunda tiene un pequeño sobrecoste de tiempo (+10 %) a cambio de ahorro de memoria.

La Propagación Profunda presenta un uso de memoria notablemente inferior:

- En GCC la Propagación Profunda consume 180 MB frente a 320 MB de la Propagación de Ondas y 250 MB de Andersen.
- En SPEC requiere sólo 170 MB frente a 290 MB de la Propagación de Ondas y 230 MB de Andersen.

La Propagación Profunda reduce el pico de memoria de uso aproximadamente 30%– 45% respecto a Andersen y especialmente frente a la de Ondas, gracias a su marcado número de nodos sin caches intermedios.

Precisión

Ambos métodos basados en inclusión, mantienen tasas de falsos positivos y negativos comparables a Andersen, entre 1–2 % para los algoritmos de propagación contra un 3–4% para Andersen, y muy superiores en precisión frente a Steensgaard (5–8 %). Con esto podemos ver que la optimización de tiempo o memoria no compromete la exactitud del análisis.

Comparación con Steensgaard

Aunque Steensgaard es ligeramente más rápido (38,9 segundos en GCC y 33,7 segundos

en SPEC), su precisión cae aproximadamente 90 %, lo que lo hace menos adecuado para optimizaciones que requieran total confianza en la detección de alias.

Criterios de selección: ¿Cuál es la mejor opción entre los algoritmos de Propagación?, para determinarlo lo hacemos diferenciando distintas circunstancias:

- Cuando el tiempo de compilación es crítico (por ejemplo en proyectos muy grandes), el algoritmo de propagación de ondas resulta la mejor opción.
- En entornos con recursos limitados (con memoria ajustada), el algoritmo de Propagación Profunda permite reducir memoria sin sacrificar precisión.
- Para casos intermedios, se podría considerar un algoritmo híbrido que active Propagación de Ondas hasta alcanzar cierto límite de uso de memoria y luego cambie a Propagación Profunda.

Conclusión

En esta investigación se ha pasado a un entorno práctico y comparativo la implementación de los algoritmos de Propagación de Ondas y Propagación Profunda de Pereira & Berlín (2009), incorporando propuestas de mejoras en los algoritmos. Los experimentos con *benchmarks* de 50 000 y 231 000 líneas de código han validado la hipótesis de partida:

La Propagación de Ondas, favorece una propagación más rápida de los conjuntos *points-to*, acortando el tiempo de análisis en un 25–27 % frente a Andersen, a costa de un aumento moderado del 40 % en el uso de memoria frente a la Profunda.

La Propagación Profunda, reduce significativamente el consumo de memoria (–25/–30 %) en comparación con Andersen y a la Propagación de Ondas, con una penalización leve de tiempo (aproximadamente +10%).

Además, ambos métodos mantienen la precisión *inclusion-based* del análisis, con tasas de falsos positivos y negativos por debajo del 2%, muy por encima de los resultados de Steensgaard.

Estas comparaciones establecen criterios para elegir la estrategia de propagación más adecuada según las restricciones de tiempo y memoria del compilador, por ende podemos decir que la elección causa efectos distintos en los compiladores, tal como se planteó en la hipótesis. Además si la elección consciente entre ambas estrategias de propagación permite optimizar tanto el rendimiento como el uso de recursos de los compiladores modernos, contribuyendo a generar código más eficiente y fiable en proyectos de gran escala.

Referencias bibliográficas

- [1] Pearce, D. J., Kelly, P. H. J., & Hankin, C. (2004). Online Cycle Detection and Difference Propagation for Pointer Analysis.
- [2] Pearce, D. J., Kelly, P. H. J., & Hankin, C. (2006). Efficient Field-Sensitive Pointer Analysis for C.
- [3] Pereira, F. M. Q., & Berlin, D. (2009). Wave Propagation and Deep Propagation for Pointer Analysis.
- [4] Hardekopf, B., & Lin, C. (2011). Flow-Sensitive Pointer Analysis for Millions of Lines of Code.
- [5] Lars Ole Andersen (1994). Program Analysis and Specialization for the C Programming Language.
- [6] Steensgaard, Bjarne (1996) Points-to Analysis in Almost Linear Time.